



Tarea 1: Pila, Cola y Tabla Hash

Hermann Pauwells Rivera

A01741456

Instituto Tecnológico y de Estudios Superiores de Monterrey, Campus Monterrey

Desarrollo de Aplicaciones Avanzadas de Ciencias Computacionales

Módulo 3: Compiladores

1 de octubre de 2026

Tarea 1: Pila, Cola y Tabla Hash

Liga de Git: <https://github.com/HermannPR/compiladores-ad26>

Qué hice

Implementé en Python 3, desde cero y sin librerías externas (solo la librería estándar), las tres estructuras que pide la tarea, cada una con sus operaciones clásicas de acceso y manipulación. Elegí Python porque es el lenguaje que planeo usar para el mini-proyecto.

- Pila (LIFO), `estructuras/pila.py`: usa una lista de Python donde el tope es el último elemento. Métodos: `push`, `pop`, `peek`, `is_empty`, `size`, `clear`, además de `len`, iteración (del tope al fondo) y `repr`. `pop` y `peek` sobre una pila vacía lanzan `IndexError` con un mensaje claro.
- Cola (FIFO), `estructuras/cola.py`: lista ligada simple con apuntadores al primero y al último nodo, así `enqueue` y `dequeue` son $O(1)$. Métodos: `enqueue`, `dequeue`, `front/peek`, `is_empty`, `size`, `clear`, `len`, iteración y `repr`. `dequeue` y `front` sobre una cola vacía lanzan `IndexError`.
- Tabla hash / diccionario, `estructuras/tabla.py`: arreglo de cubetas con encadenamiento separado (cada cubeta es una lista de nodos). Cuando el factor de carga (elementos / cubetas) pasa de 0.75 se duplican las cubetas y se hace rehash de todos los nodos. Cada nodo también está en una lista doblemente ligada que guarda el orden de inserción, por eso `keys`, `values` e `items` e iteración regresan en ese orden y borrar es $O(1)$ sobre ese orden. Sobrescribir una llave no cambia su posición; borrar y reinsertar la manda al final. Métodos: `put/insert`, `get` (con valor por defecto), `remove/delete`, `contains`, `keys`, `values`, `items`, `size`, `clear`, y los operadores `[]`, `[]=`, `del`, `in`, `len`, iteración y `repr`. El constructor acepta una función hash opcional para poder probar colisiones.

- demo.py: programa que usa las tres estructuras con ejemplos de compiladores: paréntesis balanceados con la pila, una cola de tokens a partir de la expresión $x = 3 + y1 * (42 - z)$, y una tabla de símbolos (declarar variables con tipo y valor, buscar, actualizar y borrar).
- tests/test_estructuras.py: 46 pruebas con unittest.

Uso de IA

Herramienta: Claude Code (Anthropic), como asistente de programación. Yo definí lo que necesitaba: las tres estructuras de la tarea en Python, que es el lenguaje que voy a usar en el mini-proyecto, implementadas desde cero y sin librerías, con un programa de demostración enfocado en compiladores (paréntesis balanceados, cola de tokens y tabla de símbolos), pruebas automáticas y comentarios sencillos en el código. Con esas instrucciones la herramienta generó una primera versión del código, de las pruebas y de este documento. Después revisé el código, ejecuté el demo y las 46 pruebas para comprobar cada operación, y lo subí a mi repositorio de Git.

Consultas (prompts)

Consulta 1 (instrucción de Hermann):

"asignen a un subagente a hacer la siguiente tarea. notas antes de pegar las instrucciones: tenemos que poner comentario asi como si lo hubiera hecho yo por ejemplo pila 1 e incluir metodos o lo que lo componga asi escrito como comentario si yo lo hubiera hecho, normalito. adelante"

[instrucciones de Canvas y el PDF Tar#1 ad26.pdf]

Consulta 2 (resumen de la instrucción dada al subagente):

"Implementar en Python desde cero Pila, Cola y TablaHash con sus operaciones clásicas, un programa demo con ejemplos de compiladores (paréntesis balanceados, cola de tokens, tabla de símbolos), pruebas con unittest y el documento de entrega"

Test cases

Las pruebas están en tests/test_estructuras.py (unittest). Son 46 en total y todas pasan.

Cubren casos normales, casos límite (estructura vacía, un solo elemento, muchos elementos), colisiones, redimensionado, sobrescritura, borrar y reinsertar, y orden de inserción.

Estructura	Caso (test)	Entrada / acción	Resultado esperado	Resultado
Pila	test_pila_nueva_esta_vacia	Pila()	is_empty() es True y size() es 0	OK
Pila	test_push_y_pop_lifo	push 1, 2, 3 y tres pop	salen 3, 2, 1	OK
Pila	test_peek_no_saca	push a, b y peek	regresa b y size sigue en 2	OK
Pila	test_pop_pila_vacia	pop sobre pila vacia	IndexError	OK
Pila	test_peek_pila_vacia	peek sobre pila vacia	IndexError	OK
Pila	test_un_solo_elemento	push 7 y pop	regresa 7 y queda vacia	OK
Pila	test_clear	push 1, 2 y clear	queda vacia, len 0	OK
Pila	test_muchos_elementos	10000 push, pop y peek	len 10000, pop 9999, peek 9998	OK
Pila	test_iteracion_tope_a_fondo	push 1, 2, 3 y list(pila)	[3, 2, 1]	OK
Pila	test_acepta_none_y_tipos_mezclados	push None y 'x', dos pop	salen 'x' y None, queda vacia	OK
Pila	test_repr	repr de pila con un 1	el texto contiene 1	OK
Cola	testCola_nueva_esta_vacia	Cola()	is_empty() es True y size() es 0	OK
Cola	test_enqueue_y_dequeue_fifo	enqueue 1, 2, 3 y tres dequeue	salen 1, 2, 3	OK
Cola	test_front_y_peek_no_sacan	enqueue a, b; front y peek	ambos regresan a y size sigue en 2	OK
Cola	test_dequeueCola_vacia	dequeue sobre cola vacia	IndexError	OK
Cola	test_frontCola_vacia	front sobre cola vacia	IndexError	OK
Cola	test_un_solo_elemento	enqueue 5 y dequeue	regresa 5 y	OK

			queda vacia	
Cola	test_vaciar_y_volver_a_usar	enqueue 1, dequeue, enqueue 2 y 3	contenido [2, 3], dequeue da 2	OK
Cola	test_clear	enqueue 1, 2, clear y enqueue 9	queda vacia y luego front es 9	OK
Cola	test_muchos_elementos	10000 enqueue, dequeue y front	len 10000, dequeue 0, front 1	OK
Cola	test_iteracion_en_orden_de_salida	enqueue a, b, c y list(cola)	['a', 'b', 'c'], len 3	OK
Cola	test_intercalar_enqueue_y_dequeue	enqueue 1, 2, dequeue, enqueue 3	dequeue da 1, contenido [2, 3]	OK
Cola	test_repr	repr de cola con un 1	el texto contiene 1	OK
TablaHash	test_tabla_nueva_esta_vacia	TablaHash()	len 0 y keys() vacio	OK
TablaHash	test_put_y_get	put a=1, b=2 y get	get devuelve 1 y 2, size 2	OK
TablaHash	test_insert_es_igual_que_put	insert a=1	t['a'] es 1	OK
TablaHash	test_get_con_default	get de llave inexistente, sin y con default	None y 42	OK
TablaHash	test_getitem_llave_inexistente	t['nada'] sobre tabla vacia	KeyError	OK
TablaHash	test_sobrescribir_llave	put a=1 y luego a=2	get es 2, len 1, keys ['a']	OK
TablaHash	test_remove_regresa_valor	put a=1 y remove a	regresa 1, a ya no esta, len 0	OK
TablaHash	test_remove_llave_inexistente	remove de llave que no existe	KeyError	OK
TablaHash	test_delete_y_delitem	delete('a') y del t['b']	la tabla queda vacia	OK
TablaHash	test_borrar_y_reinsertar	put a, b; remove a; put a=3	get a es 3 y keys ['b', 'a']	OK
TablaHash	test_contains	put a=None; contains, in y not in	a esta (aunque su valor sea None), b no	OK
TablaHash	test_setitem_getitem	t['x'] = 10 y t['x']	10	OK
TablaHash	test_orden_de_insercion	put z, a, m, b, y; keys, values, items	mismo orden en que se insertaron	OK
TablaHash	test_sobrescribir_no_cambia_el_orden	put a, b y luego a=99	keys ['a', 'b']	OK
TablaHash	test_orden_al_borrar_en_medio	put a..e; remove c, a, e	keys ['b', 'd']	OK
TablaHash	test_iteracion	put uno, dos, tres y list(t)	['uno', 'dos', 'tres']	OK
TablaHash	test_muchos_elementos_con_resize	1000 put con capacidad inicial 2	todos recuperables,	OK

			factor de carga ≤ 0.75 , mas cubetas, orden intacto	
TablaHash	test_colisiones_misma_cubeta	funcion hash constante, 30 llaves, remove y sobrescribir	todas se encuentran, la borrada no, len 29	OK
TablaHash	test_distintos_tipos_de_llave	llaves 1, '1', (1, 2), 2.5	cada una conserva su valor	OK
TablaHash	test_resize_despues_de_borrar	50 put, borrar pares, 50 put mas	keys impares primero, len 75, get 99 es 99	OK
TablaHash	test_clear	put a=1, clear, put b=2	a no existe, keys ['b']	OK
TablaHash	test_capacidad_invalida	TablaHash(capacidad=0)	ValueError	OK
TablaHash	test_repr	repr con a=1	TablaHash({'a': 1})	OK

Cómo ejecutar

Se necesita Python 3 (solo librería estándar). Desde la carpeta del proyecto:

- `python demo.py`
- `python -m unittest -v`